

CTA On-Time Performance

Data Pipeline — Setup Guide

A high-level reference for collecting, storing, and analysing CTA rail reliability data using the Train Tracker API and AWS.

1. Background & Goal

The CTA does not publish a downloadable on-time performance (OTP) dataset for its rail network. The monthly performance PDFs focus on operational metrics such as ridership and ADA complaints, not schedule adherence. The performance dashboard at transitchicago.com/performance shows reliability figures, but only as an interactive web UI with no export.

The goal of this pipeline is to collect raw Train Tracker API snapshots continuously, store them alongside GTFS scheduled times, and compute OTP metrics on demand — giving you a private, analysis-ready dataset that grows daily.

2. Data Sources

Train Tracker API

- Base URL: lapi.transitchicago.com/api/1.0/ttpositions.aspx
- Returns live position of every active train on a given route.
- Key fields: run number, route, next stop, predicted arrival, isDly flag, isApp flag.
- Update frequency: approximately every 60 seconds.
- Rate limit: 100,000 requests/day — polling all 8 routes every 5 min uses ~2,300/day.
- API key required — register free at transitchicago.com/developers.

GTFS Static Schedule

- Download: transitchicago.com/downloads/sch_data/google_transit.zip
- Key files: stop_times.txt (scheduled arrivals), trips.txt, routes.txt.
- Used as the 'ground truth' to compute delay versus schedule.
- Refresh every 2–4 weeks to stay current with schedule changes.

Route Codes

Code	Line	Code	Line
------	------	------	------

Red	Red Line	Org	Orange Line
Blue	Blue Line	P	Purple Line
Brn	Brown Line	Pink	Pink Line
G	Green Line	Y	Yellow Line

3. Pipeline Architecture

The pipeline has four layers. Each is independent — the collector runs continuously on AWS; the analysis layer runs on demand from your laptop.

Layer	What it does	Technology
Sources	Train Tracker API + GTFS zip	CTA public APIs
Collector	Polls API every 5 min, writes rows	Python + cron on EC2
Storage	Holds raw snapshots + schedule	SQLite on EBS
Analysis	Computes OTP, delay distributions	pandas on your laptop

4. AWS Setup

Step 1 — Launch an EC2 instance

- Instance type: t4g.nano (ARM, 2 vCPU, 512 MB RAM) — ~\$3.50/month.
- OS: Ubuntu 24.04 LTS (free, stable, well-documented).
- Storage: attach a 20 GB gp3 EBS volume — ~\$1.60/month.
- Security group: allow outbound HTTPS (port 443) for API calls. No inbound needed.
- Enable auto-recovery so the instance restarts after a hardware failure.

Step 2 — Set up the environment

- SSH into the instance and install Python 3 and pip.
- Install dependencies: requests, and optionally pandas for on-device checks.
- Clone or copy your poller script to /home/ubuntu/cta_poller/.
- Create the SQLite database file at /data/cta.db on the EBS volume.

Step 3 — Schedule the poller with cron

Add a crontab entry to run the poller every 5 minutes:

```
*/5 * * * * /usr/bin/python3 /home/ubuntu/cta_poller/poller.py >>
/var/log/cta_poll.log 2>&1
```

The poller loops over all 8 route codes, calls tpositions for each, and appends rows to the SQLite snapshots table. Total: 8 API calls per run, 2,304 per day.

Step 4 — Monitor for gaps

- Create a CloudWatch alarm on EC2 StatusCheckFailed — alerts you by email if the instance goes down.
- Optionally add a simple heartbeat: write a timestamp to S3 each poll cycle and alarm if it stops updating.
- Gaps in collection are permanent — the API has no historical endpoint.

Step 5 — Backup to S3

- Add a daily cron job that copies cta.db to an S3 bucket.
- Cost: effectively \$0 at this data volume (well under free tier thresholds).
- Command: `aws s3 cp /data/cta.db s3://your-bucket/backups/cta-$(date +%F).db`

5. Database Schema

snapshots table — raw API data

Column	Type	Description
id	INTEGER	Auto-increment primary key
captured_at	TEXT	ISO timestamp when the row was polled
route	TEXT	Route code (Red, Blue, Brn, etc.)
run	TEXT	Train run number
stop_id	TEXT	GTFS stop ID of next stop
stop_name	TEXT	Human-readable next stop name
arr_time	TEXT	Predicted arrival time from API
is_scheduled	INTEGER	1 = scheduled fallback, 0 = live prediction
is_delayed	INTEGER	1 = train flagged as delayed by CTA
is_approach	INTEGER	1 = train is approaching the stop

scheduled_arrivals table — from GTFS

Column	Type	Source file
trip_id	TEXT	stop_times.txt
stop_id	TEXT	stop_times.txt
route_id	TEXT	trips.txt (joined)
scheduled_time	TEXT	stop_times.txt → arrival_time
service_date	TEXT	Derived from calendar.txt

6. Computing OTP

Option A — isDly shortcut (simplest)

Use the `is_delayed` flag returned directly by the API. No GTFS join required. Group by route, date, and hour to get a delay rate — the percentage of observations where `is_delayed` = 1.

Limitation: this is a binary flag, not delay in minutes. A train that ran 30 minutes late counts the same as one that was 2 minutes late. Good for trend analysis; not suitable for precise OTP percentages.

Option B — Schedule comparison (precise)

Join the snapshots table to scheduled_arrivals on stop_id and service_date. Compute delay_minutes = actual_arrival - scheduled_arrival. Apply CTA's definition: on time = no more than 1 minute early and no more than 5 minutes late. Aggregate by route to get OTP %.

This requires the GTFS table to be loaded and kept current. More work upfront, but gives you the same metric CTA uses internally.

Useful analyses once you have data

- OTP % by route — which lines are most/least reliable.
- Delay rate by hour of day — morning vs. evening peak comparison.
- Day-of-week patterns — are weekends more or less reliable?
- Month-over-month trends — is reliability improving?
- Pre/post service change comparisons using GTFS refresh dates.

7. Cost Summary

Component	Choice	Monthly cost
Compute	EC2 t4g.nano	~\$3.50
Storage	20 GB EBS gp3	~\$1.60
Backups	S3 bucket	~\$0.25
Data transfer	Inbound only	\$0.00
Monitoring	CloudWatch alarm	\$0.00 (free tier)
Total		~\$5–6/month

Note: if you have a new AWS account the EC2 and EBS costs fall within the 12-month free tier, making your first year effectively \$0.

8. Alternatives Worth Knowing

Option	Cost	OTP data from	Effort
This pipeline (AWS)	~\$6/mo	Day you start	4–8 hrs setup
McFadden tracker API	\$0	June 2022	Email for key
NYC MTA open data	\$0	2020 (CSV)	Download file
511 SF Bay (GTFS archive)	\$0	Monthly archive	API token

The McFadden tracker is the fastest path to CTA-specific historical analysis — nearly 3 years of daily reliability data, no infrastructure required. Build your own pipeline only if you need bus data, want control over the raw collection, or are doing this to learn.

9. Key Gotchas

- Gaps are permanent. The API has no historical endpoint. If your poller goes down for a day, that data is gone. Set up the CloudWatch alarm before anything else.
 - GTFS station IDs have three entries per rail station (northbound stop, southbound stop, parent station). Use the parent mapid for Arrivals API calls and the directional stpid for precise platform-level data.
 - `is_scheduled = 1` means CTA has no live prediction and is falling back to the timetable. Exclude these from OTP calculations or flag them separately.
 - Unscheduled express runs are not marked as express in Train Tracker. A train skipping stops mid-route will appear as normal in the data.
 - GTFS times can exceed 24:00:00 (e.g. 25:30:00 for 1:30 AM next-day service). Handle this in your date arithmetic when computing delays.
 - The poller script must exit cleanly each run — cron will not kill a hung process from a previous cycle. Add a timeout to each API call (10 seconds is generous).
-

10. Quick-Start Checklist

- Register for a Train Tracker API key at transitchicago.com/developers.
- Launch EC2 t4g.nano in us-east-1 or us-east-2 (cheapest regions).
- Attach a 20 GB EBS gp3 volume and mount it at /data.
- Download the GTFS zip and load stop_times into SQLite.
- Deploy the poller script and test it manually once.
- Add the crontab entry for every-5-minute polling.
- Set up the CloudWatch StatusCheckFailed alarm with your email.
- Add the daily S3 backup cron job.
- Wait 2–4 weeks before drawing conclusions — you need enough data.
- Refresh the GTFS zip whenever CTA announces a schedule change.

Data sources: CTA Train Tracker API (transitchicago.com/developers), CTA GTFS (transitchicago.com/downloads), McFadden Reliability Tracker (brandonmcfadden.com/cta-reliability), NTD (transit.dot.gov/ntd).